Experiment 5: Custom Structured Output Parsing and PDF Text Extraction
Here is an exam-ready format with aim, theory, algorithm, program, sample output, and result that you can write in your Prompt Engineering Lab internal exam.

1. Aim
To design and execute an automated Applicant Tracking System (ATS) using the Template Pattern, PyMuPDF for extracting text from PDF resumes, and a Pydantic-backed JsonOutputParser to generate structured JSON output based on a resume and a job description.

2. Theory
- PyMuPDF (fitz): Extracts text from PDF resumes.
- Pydantic: Defines and validates the output schema and data types.
- JsonOutputParser: Converts LLM responses into structured JSON.
- ChatPromptTemplate: Creates prompts with system instructions and user inputs.
- Partial Variables (.partial()): Pre-fills fixed prompt instructions.
- LLM (Gemini): Extracts candidate details and evaluates job suitability.
- Template Pattern: Separates fixed instructions from dynamic inputs for reusability.

3. Algorithm
- Install the required Python libraries.
- Import the necessary modules.
- Initialize the Gemini language model using the API key.
- Define a Pydantic schema named ResumeSchema.
- Create a JsonOutputParser using the schema.
- Create a prompt template with format instructions.
- Use .partial() to pre-fill the format instructions.
- Read the PDF resume and extract its text using PyMuPDF.
- Accept the job description from the user.
- Pass the resume text and job description to the extraction chain.
- Parse the response into a JSON dictionary and display the output.

4. Program
```
# Step 1: Install libraries
!pip install langchain-google-genai langchain-core pydantic PyMuPDF

# Step 2: Import libraries
import os
import json
import fitz
from typing import List
from pydantic import BaseModel, Field
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import JsonOutputParser
from langchain_google_genai import ChatGoogleGenerativeAI
from google.colab import userdata

# Step 3: Initialize the model
os.environ["GOOGLE_API_KEY"] = userdata.get("GOOGLE_API_KEY")

llm = ChatGoogleGenerativeAI(
    model="gemini-3.1-flash-lite",
    temperature=0.0
)
```

```
# Step 4: Define the output schema
class ResumeSchema(BaseModel):
    candidate_name: str = Field(
        description="Full name; use UNKNOWN if missing"
    )
    phone_number: str = Field(
        description="Candidate's phone number"
    )
    email: str = Field(
        description="Candidate's email address"
    )
    total_years_of_experience: float = Field(
        description="Total professional experience in years"
    )
    list_of_skills: List[str] = Field(
        description="List of technical skills"
    )
    is_good_fit: bool = Field(
        description="Whether the candidate matches the job"
    )
    reason: str = Field(
        description="Reason for the job-fit decision"
    )
    summary: str = Field(
        description="Summary of career, education and projects"
    )
    highest_education: str = Field(
        description="Highest academic degree attained"  )
# Create the JSON output parser
output_parser = JsonOutputParser(
    pydantic_object=ResumeSchema
)
# Step 5: Create the prompt template
screener_prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        "You are an HR data extraction assistant. "
        "Extract resume details and evaluate the candidate "
        "against the job description. Use only supported facts. "
        "Do not invent missing information. "
        "Use UNKNOWN for missing text fields, 0.0 for "
        "unknown experience, and [] for missing skills. "
        "Follow these format instructions:\n"
        "{format_instructions}"
    ),
    (
        "human",
        "Resume Text:\n{resume_text}\n\n"
        "Job Description:\n{job_description}"
    )
])
# Pre-fill fixed format instructions
screener_prompt = screener_prompt.partial(
    format_instructions=output_parser.get_format_instructions()
)
```

```
# Assemble the chain
extraction_chain = (
    screener_prompt | llm | output_parser
)

# Step 6: Read PDF and job description
sample_pdf_path = input("Enter resume PDF path: ")
job_description = input("Enter job description: ")

# Extract text from the PDF
raw_document_text = ""

with fitz.open(sample_pdf_path) as doc:
    for page in doc:
        raw_document_text += page.get_text() + "\n"

# Step 7: Execute the chain
parsed_json_dict = extraction_chain.invoke({
    "resume_text": raw_document_text,
    "job_description": job_description
})

# Step 8: Display the result
print("\n--- Structured JSON Output ---")
print(json.dumps(parsed_json_dict, indent=2))
```

5. Sample Input
Resume PDF: resume.pdf
Job Description:
We are looking for a Python developer with knowledge
of Python, SQL, machine learning and data analysis.
Candidates should have relevant projects and experience.

6. Sample Output

```
json
{
  "candidate_name": "Rahul Sharma",
  "phone_number": "9876543210",
  "email": "rahul@example.com",
  "total_years_of_experience": 2.0,
  "list_of_skills": [
    "Python",
    "SQL",
    "Machine Learning",
    "Data Analysis"
  ],
  "is_good_fit": true,
  "reason": "The candidate has relevant Python and SQL skills that match the job
requirements.",
  "summary": "The candidate has two years of experience in software development and data
analysis, with relevant projects.",
  "highest_education": "B.Tech"
}
```